SMA Place Finder

Software Stack Documentation

Repository	`atlas-place-finder`
Application version	`1.0.0` (`package.json`, `tauri.conf.json`)
Documented commit	`cfb509b` (2026-09-28)
Generated	9 October 2026

Purpose of this document. A verified description of the technologies this project actually uses: what each one is responsible for, how the parts communicate, how the software is built and packaged, and what is required to run it. Every statement is traced to a file in the repository. Anything that could not be established from the repository is labelled rather than guessed.

Technology stack at a glance

TypeScript 5.9.3 throughout • Node.js ≥ 20 • Express 4.22.2
Next.js 15.5.22 • React 19.2.8 • Tailwind CSS 4.3.3
Tauri 2 (Rust) desktop shell • `@yao-pkg/pkg` single-file executable
In-memory cache by default, PostgreSQL 16 optional
PDFKit • sharp • archiver • Zod

No authentication layer is implemented (Section 12).

Generated from repository evidence. No credentials or secret values appear in this document.

Contents

1 Executive Summary **5**

2 Technology Selection: What, Why and Purpose **6**

3 Overall System Architecture **9**

4 Application Flow **11**
4.1 Search request lifecycle 11
4.2 Export pipeline 13

5 Frontend Technology **14**
5.1 Framework and rendering 14
5.2 Components and state 14
5.3 Styling 15
5.4 Map rendering 15
5.5 Communication with the backend 15
5.6 Accessibility and responsive design 15

6 Backend Technology **16**
6.1 Runtime and framework 16
6.2 Entry point and middleware 16
6.3 Service layer 16
6.4 Cross-cutting libraries 17
6.5 Background work 17

7 Programming Languages **18**

8 Frameworks **19**

9 Libraries and Important Dependencies **20**

10 UI Libraries and Styling **22**

11 Database and Data Layer **23**
11.1 What the data layer actually is 23
11.2 Schema 24
11.3 Real persistence: the filesystem 24

12 Authentication and Authorization **25**

13 API Technology **26**
13.1 Style and conventions 26
13.2 Endpoints 26

13.3 Request lifecycle . . . 27
13.4 External APIs consumed . . . 27

14 Build Tools and Package Manager 28
14.1 Package manager . . . 28
14.2 Compilation and bundling . . . 28
14.3 Scripts . . . 28
14.4 Build and packaging pipeline . . . 29

15 Testing Tools 30
15.1 What is configured . . . 30
15.2 Execution evidence . . . 31
15.3 What is not configured . . . 31

16 Deployment Technology and CI/CD 32
16.1 What exists . . . 32
16.2 The actual release process . . . 32
16.3 Health and observability . . . 32

17 Hosting Requirements 33

18 Important Dependencies and Risk Assessment 35

19 Master Technology Stack Table 37

20 Technology Interaction Matrix 39

21 Repository Evidence and Verification Notes 41
21.1 Files inspected . . . 41
21.2 How versions were established . . . 42
21.3 Limitations and open questions . . . 42

22 Final Summary 43
22.1 The confirmed stack . . . 43
22.2 Architectural characteristics that matter . . . 43
22.3 What could not be established . . . 43
22.4 Recommended next steps . . . 43

List of Figures

3.1 System architecture 9

4.1 Search request lifecycle 12
4.2 Export pipeline 13

11.1 Storage selection 23

13.1 API request lifecycle 27
13.2 External service dependencies 27

14.1 Build and packaging pipeline 29

List of Tables

2.1 Principal technologies, their responsibilities and the evidence for them 7

5.1 Interface components and their responsibilities 14

6.1 Backend service modules 16

7.1 Languages present in the repository 18

9.1 Backend runtime dependencies (direct) 20
9.2 Development and build dependencies 20
9.3 Frontend dependencies 21

10.1 UI technology: what is present and what is not 22

13.1 HTTP API surface (from `backend/src/routes/` and `index.ts`) 26

14.1 Scripts defined in the repository 28

15.1 Test coverage by area 30

17.1 Requirements for running the application 33

18.1 Operational dependencies and risks 35

19.1 Master technology stack 37

20.1 How the components communicate 39

21.1 Primary evidence 41

Chapter 1

Executive Summary

SMA Place Finder is a desktop research tool. The operator names a city and a kind of place — "museums in Kyoto", or simply "all" — and the application finds every matching place in that city, researches each one against open encyclopaedic sources, illustrates it with openly licensed photographs, and writes the result to disk as one folder per place containing an illustrated PDF and the photographs themselves.

The problem it solves. Assembling a usable dossier on every notable place in a city is otherwise manual work: finding the places, checking what each one is, finding a picture that may legally be reused, and writing it up. This application automates that pipeline end to end and produces a durable artefact on the operator's own disk rather than a web page.

Shape of the system. It is a **local-first client–server application**. A Node.js HTTP API performs all searching, research, imagery and document generation. A Next.js single-page interface drives it. In the packaged product both halves live inside one Windows executable: the API serves the statically exported interface itself, and a small Rust (Tauri) shell displays it in a native window. There is no cloud component — every deployment is a copy of the program running on someone's own machine.

Data. The application holds almost no state. Search results are kept in an in-memory cache so an export can refer back to a search; a PostgreSQL database may optionally be used for the same purpose and nothing else. The real output is files on disk. All substantive data is fetched live from public APIs — OpenStreetMap, Wikidata, Wikipedia, Wikimedia Commons and others — on each request.

Communication. The interface talks to the API over HTTP with JSON bodies, in a REST style. The API talks to roughly a dozen third-party HTTPS APIs. Long-running work reports progress by polling. No WebSocket, GraphQL or message broker is used.

Deployment. Two build scripts produce the shippable artefacts: a self-contained `.exe` and a Windows installer. There is no CI/CD pipeline, no application container image and no hosting platform configuration in the repository; both builds are run by hand on a developer machine (Section 16).

Chapter 2

Technology Selection: What, Why and Purpose

The repository does not record why any technology was chosen — there is no architecture decision record or design document. The "why" column below therefore describes the *technical role* each technology plays in this codebase and the practical benefit it delivers here. It is an interpretation of observed use, not a claim about the original author's intent. The "evidence" column is fact.

Table 2.1: Principal technologies, their responsibilities and the evidence for them

Technology	What it is	Responsibility here	Why it suits that responsibility	Evidence
Node.js	JavaScript runtime	Runs the whole backend: HTTP server, all research and imagery work, document generation	The workload is almost entirely waiting on remote APIs; a single-threaded event loop handles many concurrent requests without threads	`backend/package.json` `engines.node >= 20`; measured runtime `v24.18.0`
TypeScript	Typed superset of JavaScript	Every source file in backend, frontend and build scripts	A pipeline with many optional fields benefits from compile-time checking; `npm run typecheck` gates both halves	`tsconfig.json` in both packages; 100% of `src` is `.ts`/`.tsx`
Express	Minimal HTTP framework	Routing, JSON body parsing, CORS, static file serving, error handling	The API is a small set of JSON endpoints; a minimal router is sufficient and keeps the packaged binary small	`backend/src/index.ts`; three routers under `src/routes/`
Next.js	React application framework	The operator interface; also produces the static bundle the packaged binary serves	Supports both a dev server and a fully static export, so one codebase serves development and the shipped product	`frontend/next.config.mjs` (`output: 'export'` when `NEXT_EXPORT=1`)
React	UI library	All interface components	Component model suits a form plus a live-updating result list	`frontend/components/` (9 components)
Tailwind CSS	Utility-first CSS framework	All styling	Styling lives beside the markup, so the interface ships without a separate stylesheet system	`frontend/postcss.config.mjs`; `app/globals.css`

Table 2.1 continued

Technology	What it is	Responsibility here	Why it suits that responsibility	Evidence
Tauri 2	Rust desktop shell	Native window; starts and supervises the server process	Uses the WebView2 already present on Windows, so the installer is ∼28 MB rather than bundling a browser engine	`desktop/src-tauri/Cargo.toml`; `tauri.conf.json`
`@yao-pkg/pkg`	Node bundler to a single executable	Packages Node, the server and the static interface into one `.exe`	Lets the product be copied to a machine with no Node installed	`build-exe.mjs`; `backend/package.json` devDependency
PDFKit	Programmatic PDF generation	Writes the per-place PDF and the search summary	Streams a PDF without a headless browser or LaTeX toolchain	`backend/src/services/pdf.ts`
sharp	Native image processing	Normalises every downloaded photograph to JPEG	Handles formats the PDF writer cannot and re-encodes at a controlled quality	`backend/src/services/archive.ts`
archiver	ZIP writer	Builds the ZIP for browser downloads	Streams entries, so a large export needs no temporary file	`backend/src/services/archive.ts` (`zipSink`)
Zod	Schema validation	Validates every request body and query parameter	Request shapes are declared once and produce both runtime checks and types	`src/routes/*.ts`
PostgreSQL + `pg`	Relational database and driver	*Optional* shared cache of past searches	Allows several instances to share search history; the application runs without it	`src/services/cache.ts`; `docker-compose.yml`
`node:test`	Test runner built into Node	All 122 automated tests	No test framework dependency to install, version or package	`backend/package.json` `"test"` script

Chapter 3

Overall System Architecture

Figure 3.1 shows the components that exist in the repository and how they communicate. Three delivery shapes share one backend: the development servers, the packaged executable, and the desktop installer. Only the packaging differs — the server code is identical in all three.

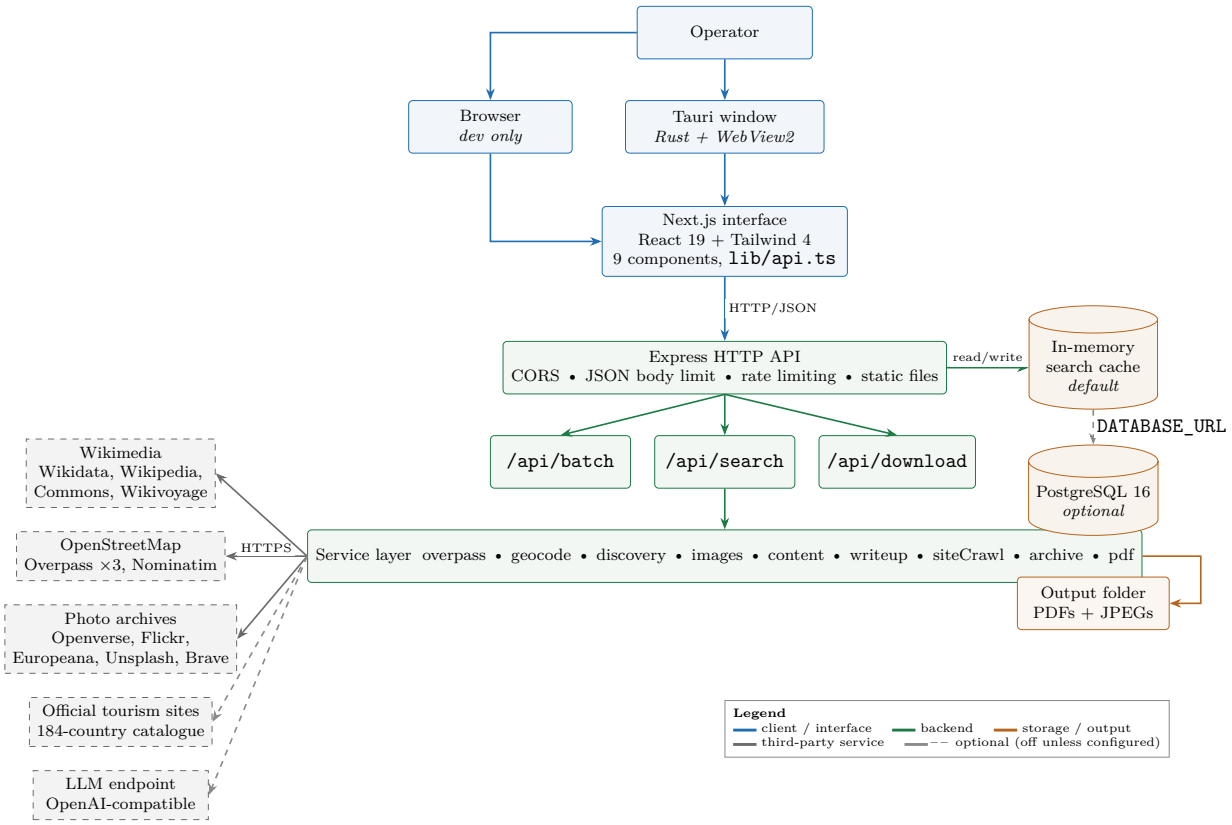


Figure 3.1: System architecture. Solid arrows are paths every run takes; dashed arrows are optional and inactive unless the corresponding setting or API key is supplied. The PostgreSQL store, the LLM endpoint, the keyed photo archives and the tourism-site crawl are all optional.

Reading the diagram. Three observations matter more than the boxes themselves.

- **One backend, three front doors.** The browser, the packaged executable and the Tauri window all speak to the same Express API over HTTP. The desktop shell does not reimplement anything; it launches the same server binary as a child process and points a WebView at it.
- **Storage is incidental.** The only persistent artefact the application is trying to produce is the output folder. The caches exist so that an export can refer back to the search that produced it.

- **Most of the work is outbound.** The service layer's job is orchestrating roughly a dozen third-party APIs. This is why availability of those services, not local compute, dominates runtime behaviour.

Chapter 4

Application Flow

4.1 Search request lifecycle

Figure 4.1 traces one search from the operator pressing *Search* to results appearing. The stage names are the literal values the backend reports through its progress channel, read from `backend/src/services/search.ts`.

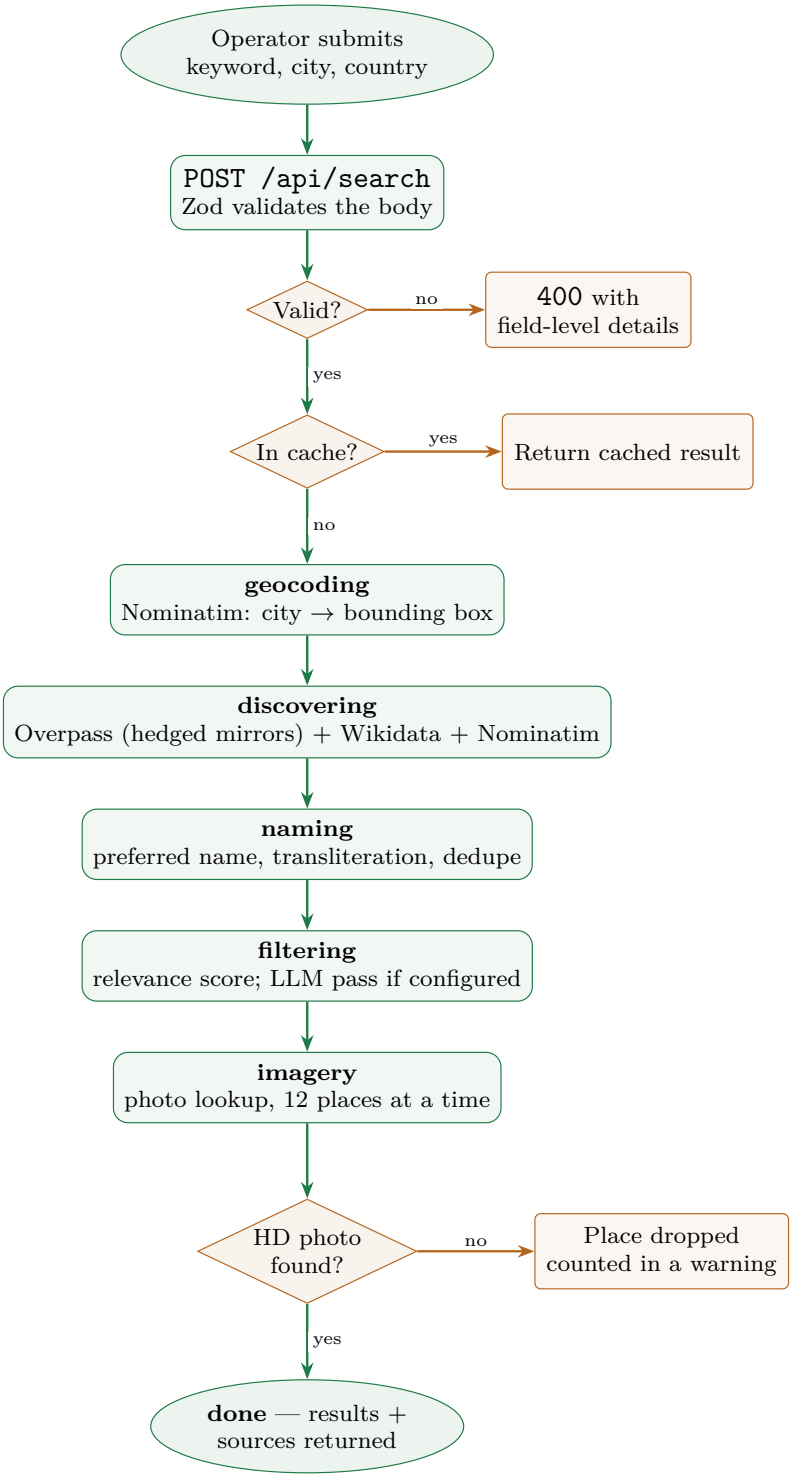


Figure 4.1: Search request lifecycle. Bold words are the progress stages the backend actually emits. The "HD photo found?" branch is active because `REQUIRE_IMAGES` defaults to true: a place with no photograph of at least `MIN_IMAGE_WIDTH` pixels is removed from the result rather than returned as text.

Note on authentication. No step in this flow checks an identity or a permission, because none exists in the codebase (Section 12). A generic flowchart would place an authentication gate after validation; this one does not, because that gate is not implemented.

4.2 Export pipeline

Export is a separate request. Figure 4.2 shows it, including the two decisions that most affect what the operator ends up with.

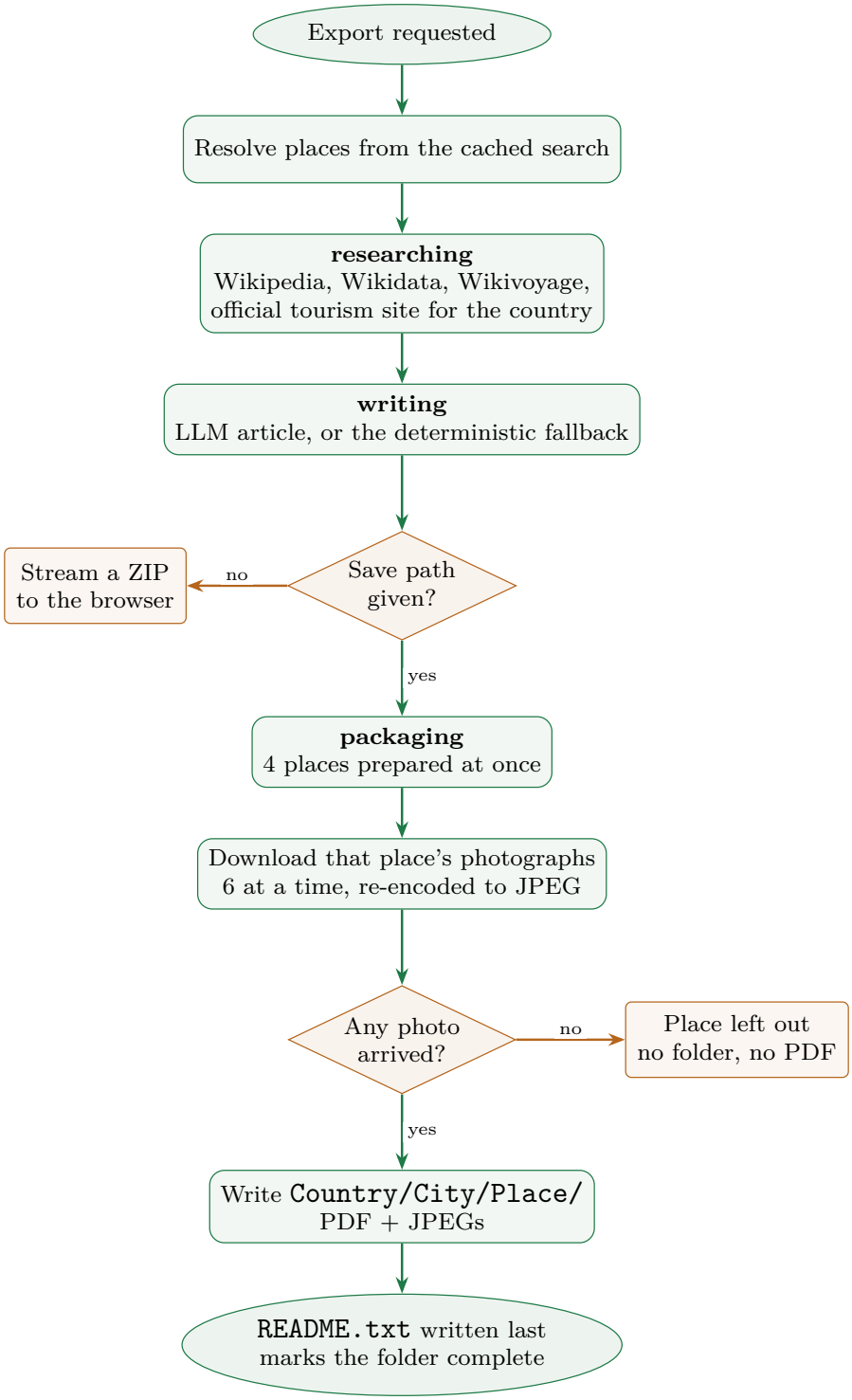


Figure 4.2: Export pipeline. A named folder receives the files directly; only a browser download produces a ZIP, because a browser cannot download a directory. A place whose photographs all fail to download is left out entirely rather than exported as a write-up with an empty folder.

Chapter 5

Frontend Technology

5.1 Framework and rendering

The interface is a **Next.js 15.5.22** application using the App Router, built on **React 19.2.8**. It has exactly one route: `frontend/app/page.tsx`, with `app/layout.tsx` supplying the document shell.

Rendering differs between development and the shipped product, and this is deliberate — `frontend/next.config.mjs` switches on an environment variable:

- **Development**: the normal Next.js dev server on port 3000, with hot reload, calling the backend on port 4000.
- **Packaged**: `build-exe.mjs` sets `NEXT_EXPORT=1`, which turns on `output: 'export'`. Next emits a fully static bundle with no Node server of its own, and the Express backend serves those files from `config.webDir`. Image optimisation is disabled in this mode because there is no optimisation server in the packaged binary.

This is why the packaged product is a single executable rather than two processes: the interface becomes static files that the API server hosts.

5.2 Components and state

Nine components live in `frontend/components/`:

Table 5.1: Interface components and their responsibilities

Component	Responsibility
`PlaceFinder.tsx`	Top-level container; owns the search result and switches between single and batch modes
`HeroSearch.tsx`	Landing state and the explanation of what the export contains
`SearchForm.tsx`	Keyword, city and country inputs, category chips, output folder picker, source list, extra source links
`ResultsPanel.tsx`	Result list, sorting, export controls, data-source disclosure
`PlaceCard.tsx`	One place: photographs, summary, outbound links
`MapView.tsx`	Map of the result area
`BatchPanel.tsx`	CSV upload, batch job control, per-region progress and retry
`SearchProgress.tsx`	Live progress for a running search
`BuildStamp.tsx`	Shows the build and commit the server reports

State management. There is **no state management library**. The `frontend/package.json` dependency list contains only `next`, `react` and `react-dom`; there is no Redux, Zustand, Jotai,

Recoil, React Query or SWR. State is held in React hooks — 51 `useState`, 18 `useEffect`, 13 `useCallback`, 10 `useRef` and 7 `useMemo` across the component tree — and passed down by props. For an application with one screen and one long-running operation, this avoids a dependency without obvious cost.

5.3 Styling

Tailwind CSS 4.3.3, wired through `@tailwindcss/postcss` in `frontend/postcss.config.mjs`. Tailwind 4 is configured in CSS rather than JavaScript: `app/globals.css` begins with `@import 'tailwindcss'` and declares the theme there. **There is no `tailwind.config.js`**, which is expected for this version and is not a missing file.

All styling is utility classes in the components plus that one stylesheet. No CSS-in-JS library, CSS modules or component library is present.

5.4 Map rendering

`MapView.tsx` embeds an `<iframe>` pointing at `https://www.openstreetmap.org/export/embed.html` with a bounding box derived from the result set. **No mapping library is used** — there is no Leaflet, Mapbox GL or Google Maps JavaScript SDK in the dependency tree. This keeps the bundle small and needs no API key, at the cost of interactive map features.

5.5 Communication with the backend

All network access goes through `frontend/lib/api.ts` using the browser `fetch` API. The base URL is `process.env.NEXT_PUBLIC_API_BASE_URL`, defaulting to `http://localhost:4000`. `frontend/lib/types.ts` mirrors the backend response shapes.

Long operations are followed by **polling**, not pushed: a search returns a progress identifier and the interface polls `/api/search/progress/:id`; batch jobs poll `/api/batch/:jobId`. No WebSocket or server-sent-events transport is present.

5.6 Accessibility and responsive design

Responsive behaviour comes from Tailwind's breakpoint utilities. **No accessibility tooling is configured** — there is no `eslint-plugin-jsx-a11y`, axe integration or automated accessibility test in the repository. This is recorded as an absence, not a defect: it means accessibility is unverified rather than known to be poor.

Chapter 6

Backend Technology

6.1 Runtime and framework

Node.js running **Express 4.22.2**. `backend/package.json` declares `engines.node >= 20`; the machine this document was produced on runs `v24.18.0`. The package is an ES module (`"type": "module"`) written entirely in TypeScript and executed through `tsx` in development.

6.2 Entry point and middleware

`backend/src/index.ts` assembles the application in this order:

1. **CORS** (`cors` 2.8.6) restricted to the configured origins.
2. **JSON body parsing** capped at `maxRequestBodyBytes` (8 MB) — the limit exists because batch requests carry an uploaded CSV.
3. **Health endpoint** `GET /api/health`, which also probes connectivity so the application notices the network returning.
4. **Three routers** mounted under `/api`.
5. **Static files** from `config.webDir` — present only in the packaged build, where this serves the exported interface.
6. **404 handler**, then a final **error handler**.

On startup the server also selects a port: it tries `PORT` (default 4000) and walks upward if the port is occupied, and if an instance is already running it opens that one instead of starting a second.

6.3 Service layer

Routes are thin; the work lives in `backend/src/services/`. The significant modules:

Table 6.1: Backend service modules

Module	Responsibility
`search.ts`	Orchestrates a search: geocode, discover, name, filter, illustrate
`overpass.ts`	Overpass queries; mirror racing, health memory, selector merging
`geocode.ts`	Nominatim lookups, city to bounding box and area id
`discovery.ts`	Extra places from Wikidata and Nominatim beyond Overpass
`images.ts`	Photo lookup across archives, HD filtering, Commons gateway fallback
`imageSources.ts`	Flickr, Europeana, Brave and Commons geo-search clients
`content.ts`	Wikipedia, Wikidata and Wikivoyage extracts per place
`siteCrawl.ts`	Shallow, robots-respecting crawl of a country's official tourism site

Module	Responsibility
`extraSources.ts`	Reduces fetched pages to text and mines passages about a place
`writeup.ts`	LLM article generation plus a deterministic fallback
`llm.ts`	OpenAI-compatible chat client; Ollama auto-detection
`translate.ts`	Name translation and transliteration via the LLM
`embeddings.ts`	Optional embeddings for semantic de-duplication
`semanticDedupe.ts`	Collapses places that are the same thing under two names
`archive.ts`	Builds the output, to a folder or a ZIP, and renders each PDF
`pdf.ts`	PDFKit document construction
`batch.ts`	Multi-city job runner with resume, retry and cancellation
`cache.ts`	Search store: in-memory, or PostgreSQL when configured
`progress.ts`	Stage reporting consumed by the interface

6.4 Cross-cutting libraries

`backend/src/lib/` holds the infrastructure the services share: `http.ts` (one fetch wrapper with per-host rate limiting, retries, timeouts and cancellation), `cancel.ts` (ambient `AbortSignal` propagation), `concurrency.ts` (bounded parallel helpers), `robots.ts` (robots.txt parsing), `publicUrl.ts` (SSRF protection for images from untrusted origins), `rateLimit.ts`, `sanitize.ts` (safe path segments), `csv.ts`, `taxonomy.ts` (keyword to OpenStreetMap tag mapping), `connectivity.ts` (offline detection) and `factCheck.ts`.

6.5 Background work

Batch processing is the only long-running work. `services/batch.ts` runs one job at a time in the same process, writing state to disk so an interrupted job can resume. There is **no job queue, scheduler or worker process** — no cron, BullMQ, Agenda or similar appears in the dependency tree.

Chapter 7

Programming Languages

Table 7.1: Languages present in the repository

Language	Where	Responsibility	Evidence
TypeScript	`backend/src`, `frontend`	All application code, both halves	`.ts`/`.tsx`; `tsconfig.json` in each package
JavaScript (ESM)	Build scripts, test files, config	Packaging, Next and PostCSS config, tests	`build-exe.mjs`, `build-desktop.mjs`, `*.test.mjs`, `next.config.mjs`
Rust	`desktop/src-tauri/src`	Desktop shell: window, child process supervision, job objects	`main.rs`; `Cargo.toml` `edition 2021`, `rust-version 1.77.2`
CSS	`frontend/app/globals.css`	Theme tokens and Tailwind import	`globals.css`
SQL	`backend/src/services/cache.ts`	Table creation and queries for the optional store	SQL literals inside `cache.ts`
JSON	Throughout	Manifests, lockfiles, Tauri configuration	`package.json`, `tauri.conf.json`
YAML	`docker-compose.yml`	PostgreSQL service definition	`docker-compose.yml`
PowerShell	`backend/src/routes/batch.ts`	Native folder-picker dialog on Windows	`execFile('powershell.exe', ...)` in `batch.ts`

Not present. No Python, Java, Go, PHP, Ruby or C# source appears in the repository. `sharp` contains native binaries, but the project itself contributes no compiled C/C++ source.

Chapter 8

Frameworks

Express 4.22.2 — backend HTTP layer. Supplies routing, middleware composition, body parsing, static file serving and centralised error handling. Used through three `Router` instances mounted under `/api`. Related files: `backend/src/index.ts`, `backend/src/routes/`. Note this is the Express 4 line, not Express 5.
Next.js 15.5.22 — frontend framework. Supplies the App Router, the development server, the production bundler and the static export used by the packaged build. Configured in `frontend/next.config.mjs`, which also declares the remote image hosts the interface is permitted to load photographs from.
React 19.2.8 — UI library. The component and hook model the entire interface is written against. Strict mode is enabled (`reactStrictMode: true`).
Tauri 2 — desktop shell framework. A Rust framework that renders a web application in the operating system's own WebView. Here it does three things: opens the window, starts the packaged server as a child process, and ties that child's lifetime to its own using a Windows job object so the server cannot outlive the window. Configured in `desktop/src-tauri/tauri.conf.json`.
Tailwind CSS 4.3.3 — styling framework. Utility classes plus a CSS-declared theme. Operates entirely at build time through PostCSS.

Testing framework. There is none, deliberately. Tests run on `node:test`, the runner built into Node, invoked as `node -import tsx -test`. See Section 15.

Chapter 9

Libraries and Important Dependencies

The project declares a small dependency surface: **8 runtime dependencies** and **11 development dependencies** in the backend, and **3 runtime dependencies** in the frontend. Versions below are the *locked* versions from `package-lock.json` (lockfile version 3), which is what actually installs.

Table 9.1: Backend runtime dependencies (direct)

Package	Declared	Locked	Why the project needs it
`express`	`^4.21.2`	`4.22.2`	HTTP server, routing and middleware. Required for startup
`cors`	`^2.8.5`	`2.8.6`	Lets the dev interface on port 3000 call the API on port 4000
`dotenv`	`^16.4.7`	`16.6.1`	Loads the `.env` that sits beside the packaged executable
`zod`	`^3.24.1`	`3.25.76`	Validates every request body and query string before it reaches a service
`pdfkit`	`^0.15.2`	`0.15.2`	Generates the per-place PDF and the search summary
`sharp`	`^0.33.5`	`0.33.5`	Converts and re-encodes every downloaded photograph. Native binary
`archiver`	`^7.0.1`	`7.0.1`	Streams the ZIP used for browser downloads
`pg`	`^8.13.1`	`8.22.0`	PostgreSQL client. Loaded lazily and only when `DATABASE_URL` is set

Table 9.2: Development and build dependencies

Package	Declared	Locked	Role
`typescript`	`^5.7.3`	`5.9.3`	Type checking and compilation (both packages)
`tsx`	`^4.19.2`	`4.23.5`	Runs TypeScript directly in development and for tests
`esbuild`	`^0.28.1`	`0.28.1`	Bundles the server into one file before packaging
`@yao-pkg/pkg`	`^6.22.0`	`6.22.0`	Produces the single-file Windows executable

Package	Declared	Locked	Role
`@types/*`	various	—	Type definitions for node, express, cors, pg, pdfkit, archiver, react

Table 9.3: Frontend dependencies

Package	Declared	Locked	Role
`next`	`^15.1.6`	`15.5.22`	Framework, dev server, bundler, static export
`react`	`^19.0.0`	`19.2.8`	UI library
`react-dom`	`^19.0.0`	`19.2.8`	DOM renderer
`tailwindcss`	`^4.0.0`	`4.3.3`	Styling (dev dependency; compiled away at build)
`@tailwindcss/postcss`	`^4.0.0`	`4.3.3`	PostCSS plugin that runs Tailwind

Transitive dependencies. Not enumerated here. The direct surface above is what the project imports; `sharp` and `next` bring the largest transitive trees, and `sharp` additionally ships platform-specific native binaries, which is why `build-exe.mjs` copies it and its dependency closure next to the executable rather than relying on the bundler.

Notably absent. No HTTP client library (the built-in `fetch` is used), no logging framework (`console` with bracketed prefixes), no ORM, no authentication library, no test framework, no state management library and no UI component library.

Chapter 10

UI Libraries and Styling

This chapter exists to record an absence precisely, because "which component library?" is usually the first question asked of a frontend.

Table 10.1: UI technology: what is present and what is not

Category	In this project	Evidence
CSS framework	Tailwind CSS 4.3.3	`postcss.config.mjs`, `app/globals.css`
Component library	**None**	No MUI, Chakra, Ant Design, shadcn/ui or Radix in `package.json`
Icon library	**None**	No icon package; inline SVG and text are used
Charting library	**None**	No Chart.js, Recharts or D3
Animation library	**None**	No Framer Motion; transitions are CSS utilities
Design system	**None formalised**	Colour tokens declared in `globals.css`
Mapping library	**None**	OpenStreetMap `<iframe>` in `MapView.tsx`
Font loading	Default stack	No `next/font` usage or webfont import found

Every control in the interface — buttons, chips, the folder picker, the progress bars, the collapsible source list — is a hand-written React component styled with Tailwind utilities. The practical consequence is a very small dependency surface and no component-library upgrade burden, in exchange for all accessibility and interaction behaviour being the project's own responsibility.

Chapter 11

Database and Data Layer

11.1 What the data layer actually is

This application has no application database in the usual sense. It stores no users, no documents and no domain records. What it has is a *search cache* with two interchangeable implementations, selected at startup in `backend/src/services/cache.ts`:

- **In-memory store (default).** A `MemoryStore` class holding recent search results in process memory. Lost on restart. This is what runs unless the operator configures otherwise.
- **PostgreSQL store (opt-in).** A `PostgresStore` used only when `DATABASE_URL` is set. The `pg` driver is imported dynamically so the dependency is never loaded otherwise.

The selection is not merely conditional, it is *forgiving*: if `DATABASE_URL` is set but the database cannot be reached, the application logs the failure and falls back to the in-memory store rather than refusing to start. This is visible in `cache.ts` and matters operationally — a database outage degrades the cache, it does not take the product down.

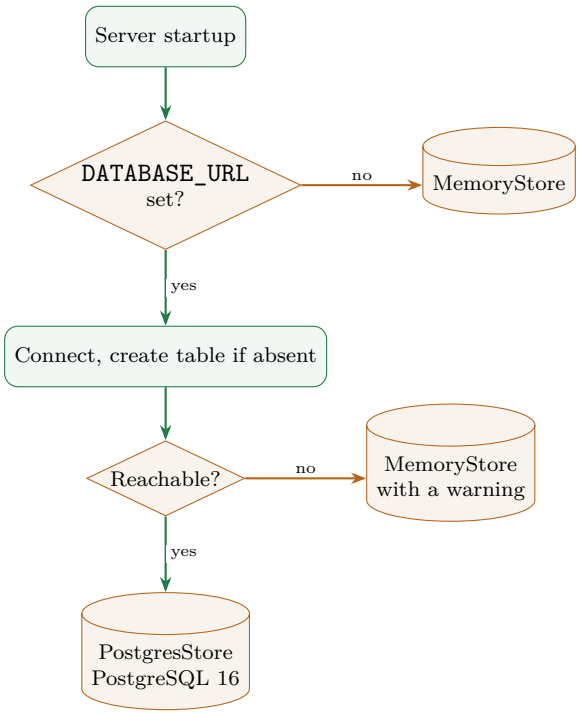


Figure 11.1: How the search store is chosen at startup. The fallback path on the right is why the application starts successfully with no database present, which is the default configuration.

11.2 Schema

The schema is created by the application itself when the PostgreSQL store initialises — there is **no migration tool and no migration directory**. No Prisma, Drizzle, TypeORM, Sequelize or Knex appears anywhere; SQL is written as string literals and executed through `pg`. There are no entities, relationships or constraints beyond the single cache table, so an entity-relationship diagram would show one box and no relationships; it is omitted as uninformative rather than fabricated.

11.3 Real persistence: the filesystem

The output the operator cares about is written to disk as a directory tree:

```
<chosen folder>/
    <Country>/
        <City>/
            <Place name>/
                <Place name>_details.pdf
                <Place name>_01.jpeg  ...
            README.txt
```

`README.txt` is written last, which makes it a completion marker: a batch resume treats a region folder containing one as finished, and rebuilds one without. Path segments are sanitised by `lib/sanitize.ts` before they touch the filesystem.

Secrets. No credentials appear in this document. The `docker-compose.yml` in the repository contains development-only credentials for a local container; those are not reproduced here and must not be used for any reachable deployment.

Chapter 12

Authentication and Authorization

No authentication or authorization is implemented in this repository.

This is a verified absence, not an omission from this document. The following were searched for across `backend/src`, `frontend/app`, `frontend/components` and `frontend/lib`: `passport`, `jsonwebtoken`, JWT handling, `bcrypt`, `argon2`, session middleware, OAuth or OIDC flows, login or signup handlers, and any `authenticate` routine. Three textual matches were found and each was inspected and ruled out:

- `services/images.ts` — the word "session" inside a code comment.
- `services/siteCrawl.ts` — `login|signin|register|account` inside a regular expression that *excludes* such pages from being crawled.
- `components/BatchPanel.tsx` — "session" in a comment and a UI label.

No route carries an authentication guard; no `req.user`, `requireAuth` or `Bearer` token check exists in `backend/src/routes/` or `index.ts`.

Outbound credentials are a different thing. The `Authorization` headers that do exist are sent *by* this application *to* third parties: `Client-ID` for Unsplash (`services/images.ts`) and `Bearer` for the LLM endpoint (`services/llm.ts`, `translate.ts`, `writeup.ts`). These authenticate the application to an external API; they do not authenticate a user to this application. The values come from environment variables and are not reproduced here.

Why this is coherent. The product is a local desktop application bound to `localhost`. The user of the program is the operator of the machine, so there is no second party to authenticate. **The consequence must be stated plainly: the API has no access control of any kind.** Anything that can reach the port can search, export and read the search history. The protections that do exist are a CORS allow-list, request-body size limits, per-route rate limiting, and SSRF defences on image fetching — these limit abuse, they do not establish identity.

If the application were ever exposed beyond localhost — published to a network interface, put behind a reverse proxy, or hosted — authentication would have to be added first. No authentication flowchart is provided, because drawing one would imply an architecture that does not exist.

Chapter 13

API Technology

13.1 Style and conventions

A **REST-style JSON API over HTTP**, served by Express and mounted entirely under `/api`. There is no GraphQL schema, no RPC framework, no WebSocket server and no API versioning segment in any path. Requests and responses are JSON, except the ZIP download, which streams binary.

Validation is uniform: every endpoint that accepts input defines a Zod schema and rejects a bad request with HTTP `400` and a list of `{field, message}` objects, so the interface can mark the offending input.

13.2 Endpoints

Table 13.1: HTTP API surface (from `backend/src/routes/` and `index.ts`)

Verb	Path	Purpose
GET	`/api/health`	Status, build stamp, connectivity, engine and LLM configuration
POST	`/api/search`	Run a search. Rate limited to 12/minute
GET	`/api/search/progress/:id`	Poll stage and counts for a running search
GET	`/api/searches/:searchId`	Retrieve a cached search result
GET	`/api/history`	Recent searches
GET	`/api/sources`	Services this install draws on; accepts `?country=` to name that country's official tourism site
GET	`/api/categories`	Category suggestions and known travel providers
GET	`/api/google-photo`	Photo proxy for the optional Google edition
POST	`/api/download`	Build the export. Rate limited to 6/minute
POST	`/api/batch`	Start a batch job from an uploaded CSV
POST	`/api/batch/preview`	Parse a CSV and report what it contains, without running
GET	`/api/batch/current`	The running job, if any
GET	`/api/batch/:jobId`	Job state, per-region rows and notes
POST	`/api/batch/:jobId/retry`	Re-queue failed regions
POST	`/api/batch/:jobId/cancel`	Cancel a running job
POST	`/api/browse-folder`	Open the native folder picker (Windows)
GET	`/api/browse-folder`	Poll the picker's result

13.3 Request lifecycle

Figure 13.1 follows one request through the layers. Note the absence of an authentication stage — see Section 12.

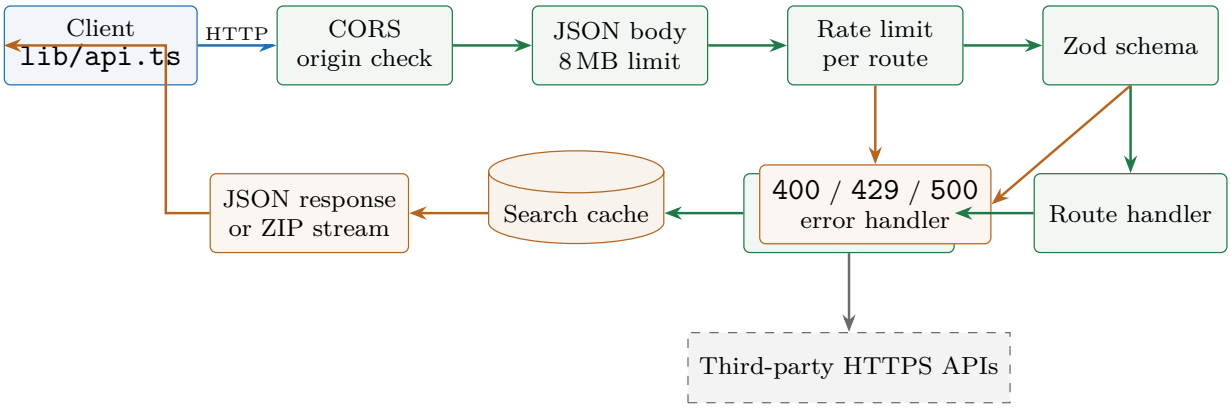


Figure 13.1: API request lifecycle. Every request passes CORS, the body-size limit, the route's rate limiter and a Zod schema before a handler sees it. There is no identity or permission stage.

13.4 External APIs consumed

Figure 13.2 groups the services the backend calls. Only the OpenStreetMap and Wikimedia groups are required; everything else is optional and inert unless configured.

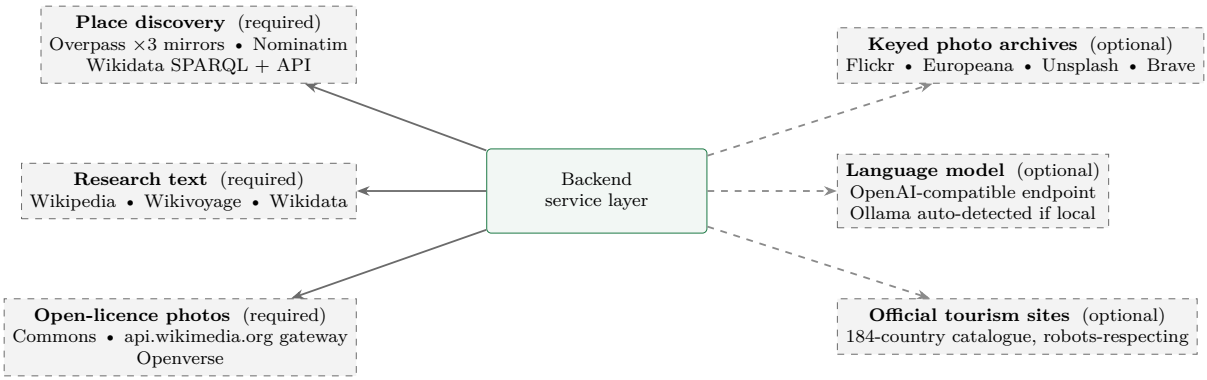


Figure 13.2: External services. Solid: contacted on every search, and the application cannot produce results without them. Dashed: skipped silently when the corresponding key or setting is absent.

API documentation. There is **no OpenAPI/Swagger specification** and no generated API documentation in the repository. Table 13.1 was compiled by reading the route files.

Chapter 14

Build Tools and Package Manager

14.1 Package manager

npm. Both packages carry `package-lock.json` at lockfile version 3; the npm in use when this document was produced was `11.16.0`. A stray `pnpm-lock.yaml` exists in the *parent* directory above the project root — Next.js warns about it when inferring its workspace root — but the project itself is installed with npm. There is no workspaces configuration; the root `package.json` installs each package with `npm -prefix <dir> install`.

14.2 Compilation and bundling

- **TypeScript 5.9.3** — `target ES2022`, `strict: true` in both packages; `module NodeNext` in the backend and `esnext` in the frontend.
- **tsx 4.23.5** — runs TypeScript directly in development and under the test runner, so there is no build step while developing the backend.
- **esbuild 0.28.1** — bundles the server into a single CommonJS file in `pkg-stage/` before packaging.
- **Next.js bundler** — builds the interface; with `NEXT_EXPORT=1` it emits a static site instead of a server bundle.
- **@yao-pkg/pkg 6.22.0** — embeds Node plus the bundled server and the static interface into one `.exe`. A maintained fork of the discontinued `vercel/pkg`.
- **Cargo / rustc** — compiles the Tauri shell. `Cargo.toml` requires `rust-version 1.77.2`; the toolchain used here was `1.98.1`. Release profile: `lto = true`, `opt-level = "s"`, `codegen-units = 1`, `panic = "abort"`, `strip = true`.

14.3 Scripts

Table 14.1: Scripts defined in the repository

Command	Effect
`npm run setup`	Installs both packages
`npm run dev:backend`	`tsx watch src/index.ts` — API on port 4000
`npm run dev:frontend`	`next dev` — interface on port 3000
`npm run build`	`tsc` for the backend, `next build` for the frontend
`npm run typecheck`	Type-checks both packages; no emit
`npm test` (backend)	`node -import tsx -test src/**/*.test.mjs`
`node build-exe.mjs`	Produces both editions of the standalone executable

`node build-desktop.mjs`	Produces the Windows installer

Linting and formatting. `frontend/package.json` defines a `lint` script calling `next lint`, but **no ESLint configuration file and no ESLint dependency exist** in the repository, so that script is not usable as written. There is no Prettier configuration and no `.editorconfig`. Code style is maintained by convention.

14.4 Build and packaging pipeline

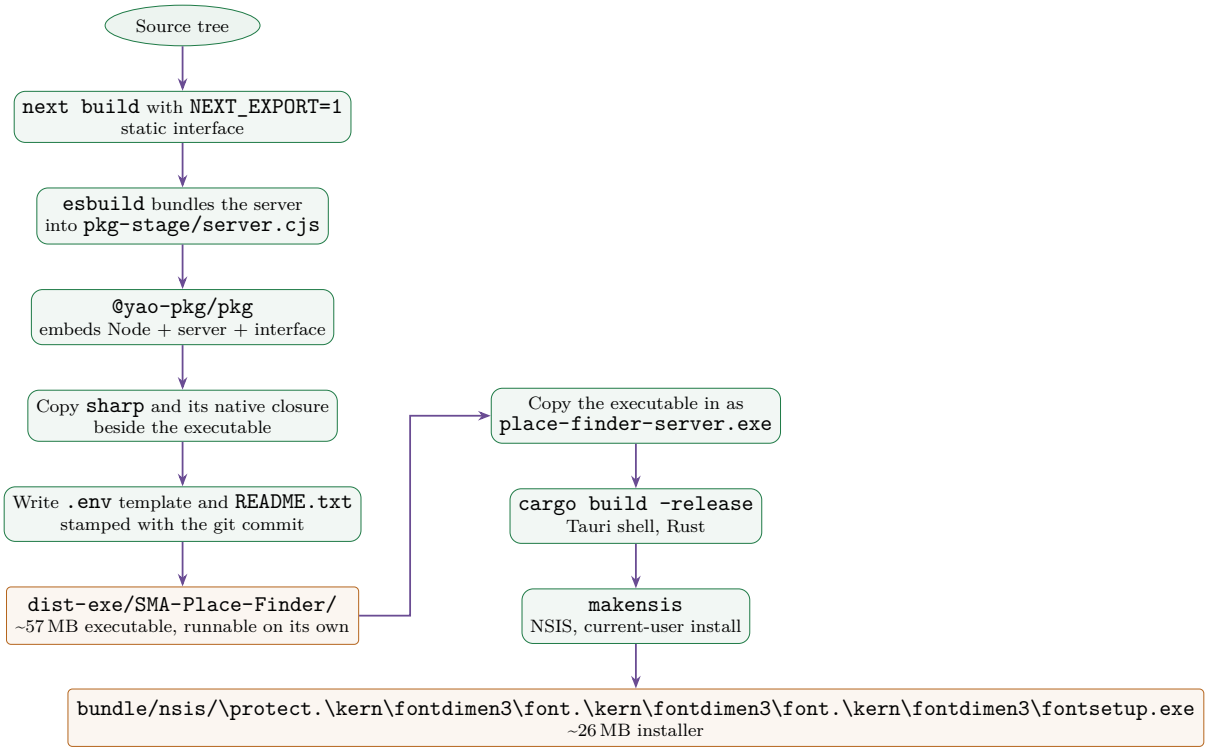


Figure 14.1: Build and packaging. The installer does not rebuild the server: it embeds the same executable the standalone edition ships, so both artefacts contain byte-identical server code.

`build-exe.mjs` stamps each build with `git rev-parse -short HEAD` and appends `-dirty` when `git status -porcelain` is non-empty, so a build made from uncommitted work identifies itself at runtime through `/api/health`.

Chapter 15

Testing Tools

15.1 What is configured

`node:test` — the test runner built into Node — driven by `node -import tsx -test src/**/*.test.mjs`. Assertions use `node:assert/strict`. There is **no test framework dependency**: no Jest, Vitest, Mocha, AVA or Jasmine appears in either `package.json`.

Table 15.1: Test coverage by area

Test file	What it covers
`lib/robots.test.mjs`	robots.txt parsing: group selection, longest-match, wildcards, `Crawl-delay`
`lib/publicUrl.test.mjs`	SSRF defences: private address ranges, URL vetting
`lib/connectivity.test.mjs`	Offline detection and recovery
`lib/factCheck.test.mjs`	Stripping unsupported opening-hours claims from generated text
`lib/llmText.test.mjs`	Repairing malformed model output
`lib/travelLinks.test.mjs`	Outbound deep-link construction
`services/overpass.test.mjs`	Selector merging; proves no tag is narrowed away
`services/siteCrawl.test.mjs`	Link extraction and scoring; tourism-source catalogue integrity
`services/hdImages.test.mjs`	HD threshold and illustrated-place selection
`services/commonsGateway.test.mjs`	Commons gateway query construction
`services/folderOutput.test.mjs`	Folder output, no-photo exclusion, non-destructive writes
`services/fallbackWriteup.test.mjs`	Write-up enrichment from map data; invents nothing when data is absent
`services/writeup.test.mjs`	Tolerant parsing of real model output
`services/batchRecovery.test.mjs`	Batch resume behaviour

15.2 Execution evidence

The suite was executed while preparing this document, from `backend/`:

> `npm test` → **tests 122, suites 16, pass 122, fail 0**

Both type checks were also run: `npm run typecheck` reports **zero errors** across backend and frontend.

15.3 What is not configured

No integration, end-to-end, API, UI or component testing exists: no Playwright, Cypress, Puppeteer, Supertest or Testing Library. **No coverage tool** is configured, so the 122 tests cannot be expressed as a coverage percentage — they are unit tests over the library and service functions listed above, and the HTTP layer, the React components and the Rust shell are untested. There is no static analysis beyond the TypeScript compiler.

`backend/src/eval/` contains `harvest.ts` and `run.ts`, invoked by `npm run eval`. These are a quality-evaluation harness for search output, not an automated test suite.

Chapter 16

Deployment Technology and CI/CD

16.1 What exists

There is no CI/CD pipeline. The repository contains no `.github/workflows/`, no GitLab CI, Jenkins, CircleCI or Azure Pipelines configuration. Nothing builds, tests or releases automatically.

There is no container image for the application. No `Dockerfile` exists anywhere. The single `docker-compose.yml` defines *one* service — `postgres:16-alpine`, with a health check and a named volume — and is a convenience for running the optional database locally. It does not deploy this application.

There is no infrastructure-as-code, reverse proxy or web server configuration — no Terraform, Pulumi, Kubernetes manifest, nginx or Apache configuration, and no platform file for Vercel, Netlify, Railway, Fly or similar.

16.2 The actual release process

Deployment is **manual packaging and distribution**, performed on a developer machine:

1. Commit the work, so the build stamps a clean commit rather than `-dirty`.
2. `node build-exe.mjs` — produces `dist-exe/SMA-Place-Finder/` (OSM edition) and `dist-exe/SMA-Place-Finder-Google/`.
3. `node build-desktop.mjs` — embeds that executable and produces the NSIS installer.
4. Copy the folder, or hand over the installer, to the target machine.
5. Verify by starting it and reading `/api/health`, which reports the build date and commit.

Runtime distribution model. The product is installed per user (`nsis.installMode: currentUser`) and runs on the operator's own machine. WebView2 is handled by `webviewInstallMode: embedBootstrapper`, so the installer fetches the runtime if the machine lacks it.

Code signing. Unknown — no signing configuration or certificate reference appears in the repository. Unsigned Windows installers raise a SmartScreen warning on first run.

16.3 Health and observability

`GET /api/health` is the only health surface and is used three ways: the desktop shell polls it to know when the server is ready, the running server uses it to detect an instance that is already running, and the interface displays its build stamp. Logging is `console` output with bracketed prefixes. There is **no structured logging, metrics endpoint, tracing or error-reporting integration**.

Chapter 17

Hosting Requirements

Each requirement is labelled: **Verified** established by project files or by execution; **Derived** reasonably inferred from the implementation; **Recommended** an operational suggestion, not a project requirement; **Unknown** insufficient evidence.

Table 17.1: Requirements for running the application

Requirement	Status	Detail and evidence
Operating system (packaged)	**Verified**	Windows x64. `targets: ["nsis"]`, `x86_64-pc-windows-msvc`, and a PowerShell folder picker
Operating system (source)	**Derived**	Backend and frontend are portable Node; only the packaging and the folder picker are Windows-specific
Node.js version	**Verified**	`engines.node >= 20`; verified running on `v24.18.0`. Not needed for the packaged build — Node is embedded
Rust toolchain	**Verified**	Build-time only. `rust-version 1.77.2` declared; `1.98.1` used here
WebView2	**Verified**	Required by the Tauri shell; the installer bootstraps it
Disk, installed	**Verified**	~57 MB executable, ~26 MB installer (measured)
Disk, output	**Derived**	Grows without bound; the operator chooses the folder. A single place is one PDF plus up to 10 photographs
Memory	**Derived**	Driven by export concurrency: 4 places in flight, each holding its photographs — on the order of 100 MB at default settings, plus Node and WebView2
CPU	**Unknown**	No measurement exists. The workload is dominated by waiting on remote APIs, not computation
Network	**Verified**	Outbound HTTPS to the services in Figure 13.2 is mandatory; with no connectivity the application detects the outage and waits
Inbound ports	**Verified**	TCP 4000 by default, walking upward if occupied; 3000 for the dev interface only. Bound for local use
TLS / HTTPS	**Verified**	Not applicable inbound — the server is plain HTTP on localhost. All outbound calls are HTTPS
Domain / DNS	*N/A*	No hosted deployment exists
Database	**Verified**	Optional. Without `DATABASE_URL` an in-memory cache is used; PostgreSQL 16 is what `docker-compose.yml` provides

Requirement	Status	Detail and evidence
Environment variables	**Verified**	All optional with defaults. The packaged build ships a commented `.env` beside the executable
API keys	**Verified**	All optional: Flickr, Europeana, Unsplash, Brave, Google Maps, LLM. Absent keys disable that source silently
Language model	**Verified**	Optional. Without it, write-ups use the deterministic fallback. Ollama is auto-detected when present
Persistent storage	**Derived**	The output folder must persist; nothing else does
Logging / monitoring	**Recommended**	Only console logging exists. Any retention or alerting would have to be added
Backup	**Recommended**	The output folder is the only artefact worth backing up; the caches are disposable
Production capacity	**Unknown**	**No load testing has been performed.** No figure for concurrent users or throughput can be given from this repository

On sizing. No CPU, RAM or bandwidth figure is stated beyond the measured artefact sizes and the derived memory behaviour above. The application has never been load tested, and inventing a server size would be a guess presented as a requirement. Establishing production capacity requires measuring a representative batch run on target hardware.

Chapter 18

Important Dependencies and Risk Assessment

Table 18.1: Operational dependencies and risks

Area	Severity	Assessment
Required for startup	—	`express`, `cors`, `dotenv`, `zod`. All small and stable
`sharp`	Elevated	A native binary that must match the platform. `build-exe.mjs` copies it and its closure beside the executable precisely because the bundler cannot embed it. If it fails to load the application still runs, but non-JPEG images are skipped — the code handles this and says so in the export notes
Overpass API	**High**	The primary source of places, run by volunteers with no availability guarantee. Mitigated in code: three mirrors raced with a hedge, failing mirrors parked for five minutes, and an empty answer never allowed to win
Nominatim	**High**	Geocoding; without it a city cannot be resolved and the search cannot start. Single endpoint, no fallback configured
Wikimedia	**High**	Photographs and research text. Partially mitigated: when `commons.wikimedia.org` is unreachable the code falls back to `api.wikimedia.org`. Verified behaviour, not theoretical
Third-party rate limits	Medium	Public endpoints throttle aggressive use. `lib/http.ts` applies per-host pacing; sustained batch runs can still be throttled
No authentication	**High if exposed**	Acceptable while bound to localhost; unacceptable on any reachable interface. See Section 12
No CI/CD	Medium	Nothing enforces that tests pass or types check before a build is produced and distributed. The checks exist; running them is manual
Unsigned installer	Medium	SmartScreen warns on first run; users must be told to expect it

Area	Severity	Assessment
Single-process batch	Medium	One batch job at a time in the API process. Job state is written to disk so a crash can resume, but a crash still interrupts the run
`pkg` ecosystem	Low–Medium	`@yao-pkg/pkg` is a community fork of the discontinued `vercel/pkg`. It works here, but the packaging approach depends on that fork continuing
Known vulnerabilities	**Unknown**	**No audit was run** as part of this document. No claim is made that any dependency is or is not vulnerable. Run `npm audit` to establish this
Dependency currency	**Verified**	Locked versions are recent — Express 4.22.2, Next 15.5.22, React 19.2.8, TypeScript 5.9.3. Express 4 is the previous major line; Express 5 is released

Single points of failure. Three, all external: Nominatim (no fallback), Overpass (mitigated by mirror racing), and Wikimedia (partially mitigated by the gateway). Everything internal degrades rather than fails — a missing database, a missing LLM, a missing API key and a missing `sharp` all have handled paths.

Chapter 19

Master Technology Stack Table

Versions are the *locked* versions from `package-lock.json` unless noted. "Observed" marks a version measured on the machine used to produce this document rather than declared by the project.

Table 19.1: Master technology stack

Category	Technology	Version	Purpose
Frontend framework	Next.js	`15.5.22`	Interface, dev server, static export
UI library	React	`19.2.8`	Components and hooks
	React DOM	`19.2.8`	DOM rendering
Styling	Tailwind CSS	`4.3.3`	All interface styling
	`@tailwindcss/postcss`	`4.3.3`	Build-time Tailwind processing
Backend runtime	Node.js	`>=20` declared; `24.18.0` observed	Executes the API and all services
Backend framework	Express	`4.22.2`	HTTP routing and middleware
Language	TypeScript	`5.9.3`	All application source
	Rust	`>=1.77.2` declared; `1.98.1` observed	Desktop shell
Desktop shell	Tauri	`2` (major, `Cargo.toml`)	Native window and server supervision
	`windows-sys`	`0.59`	Job objects, so the server cannot outlive the shell
	`ureq`	`2`	Health polling from the shell
Validation	Zod	`3.25.76`	Request schemas
PDF generation	PDFKit	`0.15.2`	Per-place and summary PDFs
Image processing	sharp	`0.33.5`	JPEG conversion and re-encoding
Archiving	archiver	`7.0.1`	ZIP for browser downloads
HTTP middleware	cors	`2.8.6`	Cross-origin access for the dev interface
Configuration	dotenv	`16.6.1`	Loads the `.env` beside the executable
Database (optional)	PostgreSQL	`16`	Optional shared search cache; image `postgres:16-alpine`

Category	Technology	Version	Purpose
Database driver	`pg`	`8.22.0`	Client, loaded only when configured
Default storage	In-memory store	*N/A*	Search cache; the default
Authentication	**None implemented**	*N/A*	No identity or access control exists
API style	REST over HTTP, JSON	*N/A*	Client–server communication
Build tool	esbuild	`0.28.1`	Server bundling before packaging
	`@yao-pkg/pkg`	`6.22.0`	Single-file executable
	Next.js bundler	`15.5.22`	Interface build and static export
Dev runner	tsx	`4.23.5`	Runs TypeScript directly
Package manager	npm	`11.16.0` observed; lockfile v3	Dependency management
Testing	`node:test`	Node built-in	122 unit tests, 16 suites, all passing
Installer	NSIS via Tauri bundler	*Not Verified*	Windows installer, current-user
CI/CD	**None**	*N/A*	No automated pipeline exists
Containerisation	Docker Compose (database only)	*Not Verified*	Local PostgreSQL convenience only
Hosting	Local desktop	*N/A*	No hosted deployment exists

Chapter 20

Technology Interaction Matrix

Table 20.1: How the components communicate

Source	Target	Method	Purpose	Evidence
Next.js interface	Express API	HTTP/JSON `fetch`	All operations	`frontend/lib/api.ts`
Interface	`/api/search/progress/:id`	HTTP polling	Live search progress	`SearchProgress.tsx`
Interface	`/api/batch/:jobId`	HTTP polling	Batch job state	`BatchPanel.tsx`
Interface	openstreetmap.org	`<iframe>` embed	Map display	`MapView.tsx`
Tauri shell	Packaged server	Child process + TCP	Starts it, polls `/api/health`	`desktop/src-tauri/src/main.rs`
Tauri shell	Windows kernel	Job object API	Server cannot outlive the window	`windows-sys`, `main.rs`
Express routes	Services	In-process calls	Business logic	`routes/`, `services/`
Services	Overpass mirrors	HTTPS POST	Place discovery	`services/overpass.ts`
Services	Nominatim	HTTPS GET	Geocoding	`services/geocode.ts`
Services	Wikidata / Wikipedia / Wikivoyage	HTTPS GET	Research text and facts	`services/content.ts`
Services	Commons, `api.wikimedia.org`	HTTPS GET	Photographs, with gateway fallback	`services/images.ts`
Services	Flickr, Europeana, Unsplash, Brave	HTTPS GET	Extra photographs, when keyed	`services/imageSources.ts`
Services	Official tourism sites	HTTPS GET, robots-respecting	Write-up material	`services/siteCrawl.ts`
Services	LLM endpoint	HTTPS POST, OpenAI-compatible	Filtering, translation, articles	`services/llm.ts`

Source	Target	Method	Purpose	Evidence
Services	Search cache	In-process or SQL	Result reuse between search and export	`services/cache.ts`
`cache.ts`	PostgreSQL	TCP 5432 via `pg`	Optional shared cache	`services/cache.ts`
`archive.ts`	Filesystem	`fs` writes	The output folder	`services/archive.ts`
`batch.ts`	PowerShell	`execFile`	Native folder picker	`routes/batch.ts`

Chapter 21

Repository Evidence and Verification Notes

21.1 Files inspected

Table 21.1: Primary evidence

File	What it established
`package.json` (root)	Workspace scripts; npm with `-prefix`
`backend/package.json`	Runtime and dev dependencies; `engines.node`; test command
`backend/package-lock.json`	Locked backend versions (lockfile v3)
`frontend/package.json`	Next, React, Tailwind; absence of state and UI libraries
`frontend/package-lock.json`	Locked frontend versions
`backend/tsconfig.json`	`ES2022`, `NodeNext`, `strict`
`frontend/tsconfig.json`	`ES2022`, `esnext`, `strict`
`backend/src/index.ts`	Middleware order, routers, static serving, port selection
`backend/src/config.ts`	All settings, defaults, limits, external endpoints
`backend/src/routes/*.ts`	Endpoint inventory, Zod validation, rate limits
`backend/src/services/cache.ts`	Storage strategy and the PostgreSQL fallback
`backend/src/services/archive.ts`	Output structure, folder/ZIP sinks, photo gate
`backend/src/services/overpass.ts`	Mirror racing, health memory, selector merging
`frontend/next.config.mjs`	Static export switch; permitted remote image hosts
`frontend/postcss.config.mjs`	Tailwind 4 via PostCSS
`frontend/app/globals.css`	CSS-first Tailwind configuration
`frontend/lib/api.ts`	API base URL and the polling model
`desktop/src-tauri/Cargo.toml`	Rust edition, `rust-version`, Tauri and `windows-sys`
`desktop/src-tauri/tauri.conf.json`	Bundle targets, resources, WebView2, install mode
`build-exe.mjs`	Packaging pipeline, `.env` template, build stamping
`build-desktop.mjs`	Installer pipeline and server embedding
`docker-compose.yml`	PostgreSQL 16 service; absence of an application image

21.2 How versions were established

- **npm packages** — read from the `packages` map of each `package-lock.json`, not from the declared ranges in `package.json`. Both are shown in Section 9 so the difference is visible.
- **Node and npm** — `node -version` and `npm -version` on the machine used here, labelled "observed". The project only declares `>=20`.
- **Rust** — `rustc -version` observed; `Cargo.toml` declares the minimum.
- **Tauri, ureq, windows-sys** — major versions from `Cargo.toml`. **`Cargo.lock` was not consulted**, so exact resolved crate versions are *Not Verified*.
- **PostgreSQL** — the image tag `postgres:16-alpine`.

21.3 Limitations and open questions

- **No design rationale exists.** There is no architecture decision record. Every "why" in Chapter 2 is an interpretation of observed use.
- **Exact Rust crate versions** were not resolved from `Cargo.lock`.
- **No dependency audit** was performed; no vulnerability claim is made either way.
- **No coverage measurement** exists, so the 122 passing tests cannot be expressed as a proportion of the code.
- **No load testing** exists, so no capacity figure is given.
- **The `lint` script is not runnable** as written — `next lint` is defined but ESLint is neither configured nor installed.
- **The Google edition** is built from the same source with `PLACE_SOURCE=google`; its behaviour was not exercised while preparing this document.

Chapter 22

Final Summary

22.1 The confirmed stack

A TypeScript application, end to end. An **Express** API on **Node.js** does all the work; a **Next.js**/**React** interface with **Tailwind** styling drives it; a **Tauri** shell in **Rust** presents it as a Windows desktop program. Documents are produced with **PDFKit**, images normalised with **sharp**, archives written with **archiver**, and every request validated with **Zod**. Tests run on Node's own runner — 122 of them, all passing.

22.2 Architectural characteristics that matter

1. **Local-first.** No hosted component. Each deployment is a copy running on someone's machine, which is why there is no authentication and why that is defensible.
2. **Orchestration, not computation.** The substance is coordinating a dozen public APIs. Availability and rate limits of those services dominate behaviour far more than local resources.
3. **Degradation over failure.** Missing database, missing LLM, missing API key, blocked Commons host, failed `sharp` load, lost connectivity — each has a handled path. The application narrows what it produces instead of refusing to run.
4. **The filesystem is the product.** Caches are disposable; the output folder is the artefact.
5. **A deliberately small dependency surface.** Eleven direct runtime dependencies across both halves, no framework for testing, state, UI components or data access.

22.3 What could not be established

Exact Rust crate versions; dependency vulnerability status; test coverage; production capacity; and the original rationale for any technology choice.

22.4 Recommended next steps

These are suggestions, not findings. Nothing below is a defect in the current product.

1. **Add continuous integration.** The checks already exist (`npm test`, `npm run typecheck`); nothing enforces them before a build is distributed. This is the cheapest meaningful improvement available.
2. **Run `npm audit`** and record the result, so dependency risk is known rather than unlabelled.
3. **Make the `lint` script real** — install and configure ESLint, or remove the script so it does not imply a check that cannot run.
4. **Add authentication before any network exposure.** Only necessary if the deployment model changes; today it would be unnecessary complexity.
5. **Consider code signing** to remove the SmartScreen warning.

6. **Measure a representative batch run** on target hardware to replace the **Unknown** capacity entries in Table 17.1 with figures.
7. **Record a short architecture decision log**, so future readers inherit the reasoning this document had to infer.

---

*Every technology, version and component above was read from the repository at commit `cfb509b`. Items that could not be verified are marked Not Verified, **Unknown** or stated as absent. No credentials or secret values appear in this document.*